

Summary: Advanced DAX Functions

SESSION OVERVIEW:

By the end of this session, students will be able to:

- Grasp Date Time functions and Time Intelligence Functions.
- Write complex DAX queries using advanced DAX functions.

KEY TOPICS AND EXAMPLES:

Note: In this session we will use [this dataset](#) to practice the concepts.

Note: Before using date functions, we must ensure that the date columns in our dataset are read properly as date data type. Further we need to ensure that the format is correct. For example, we may get data from Singapore in a MM-DD-YYYY format but our Power BI system setting (United States) may recognise it in a different format (e.g. DD-MM-YYYY). If it does so then our values will be incorrect or it will throw an error. Hence it is important to ensure that we change the Locale in the Power BI settings.

To do this, go to File → Options & Settings → Options → Regional Settings → in the Locale for Import box select the desired locale: e.g. English (United States).

This will determine how Power BI interprets the dates in our dataset.

1. DAX Date Time Functions

a. TODAY

It returns the current date.

Syntax: TODAY()

b. NOW

It returns the current date and time.

Syntax: NOW()

c. DATE

It returns a date value given the specified year, month, and day.

Syntax: DATE(<year>, <month>, <day>)

d. TIME

It returns a time value based on the specified hour, minute, and second.

Syntax: TIME(<hour>, <minute>, <second>)

e. YEAR

It returns the year component from a date.

Syntax: YEAR(<date>)

f. MONTH

It returns the month component from a date.

Syntax: MONTH(<date>)

g. DAY

It returns the day component from a date.

Syntax: DAY(<date>)

h. HOUR, MINUTE, SECOND

These functions return the hour component of a timestamp (0 to 23), minute component of a timestamp (0 to 59) and second component of a timestamp (0 to 59) respectively.

Syntax: HOUR(<time>), MINUTE(<time>), SECOND(<time>)

i. WEEKDAY

It returns the day of the week as a number from 1 to 7. 1 represents Sunday, 2 represents Monday, and so on, up to 7 for Saturday.

Syntax: WEEKDAY(<date>, [<return_type>])

Note: Here [<return_type>] is optional and can be used to specify the numbering convention for the day of the week. These are the various numbering conventions:

- 1: Numbers the days from Sunday (1) to Saturday (7).
- 2: Numbers the days from Monday (1) to Sunday (7).
- 3: Numbers the days from Monday (0) to Sunday (6).

j. EOMONTH

It returns the last day of the month after adding the specified number of months to the start date.

Syntax: EOMONTH(<start_date>, <months>)

k. DATEDIFF

It returns the difference between two dates in the specified interval.

Syntax: DATEDIFF(<start_date>, <end_date>, <interval>)

Here, the <interval> is the interval to use when comparing dates. The value can be one of the following:

- SECOND
- MINUTE
- HOUR
- DAY
- WEEK
- MONTH
- QUARTER
- YEAR

I. DATEADD

It returns a new date after adding or subtracting a specified number of intervals to the start date.

Syntax: DATEADD(<start_date>, <number>, <interval>)

The list of <interval> values are given below:

- year
- quarter
- month
- day
-

Note: A DateTime value can be stored as a **decimal number**, where:

- The **integer part** represents the **date**.
- The **decimal part** represents the **time**.

Example:

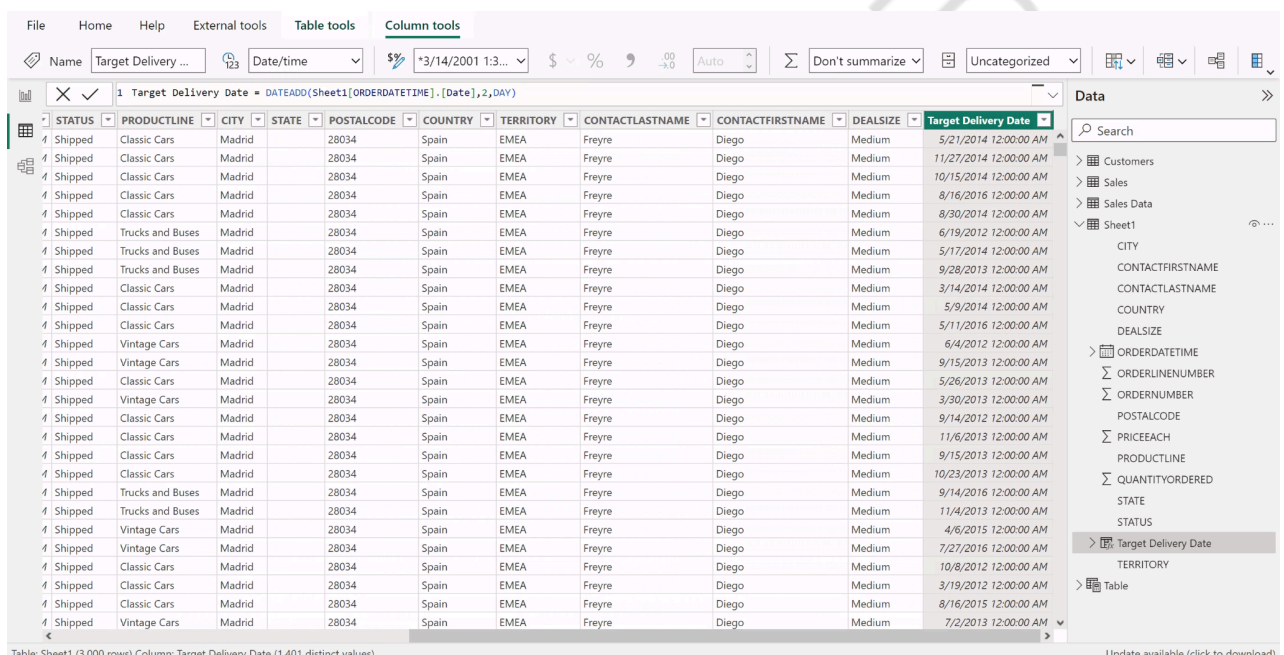
2024-03-04 12:00 PM → 45264.5 (where **.5** represents **half a day**, i.e., **12 hours**).

This concept is crucial when performing calculations like adding hours, days, or handling time-based transformations.

For example, in our [Vehicle Orders](#) dataset, we need to define a target delivery date. Our company targets to deliver orders within 48 hours to our users. If we fail to do so, we will breach the delivery promise to our users. Let us find in how many cases we have breached the delivery promise. For this there are multiple ways, one of which is to add 48 hours (2 days) to the Order Date and compare this value with the actual delivery date. If the difference (Promised Delivery Date - Actual Delivery Date) is positive then we have delivered on our promise otherwise we have failed to do so.

Here the following formula can be used:

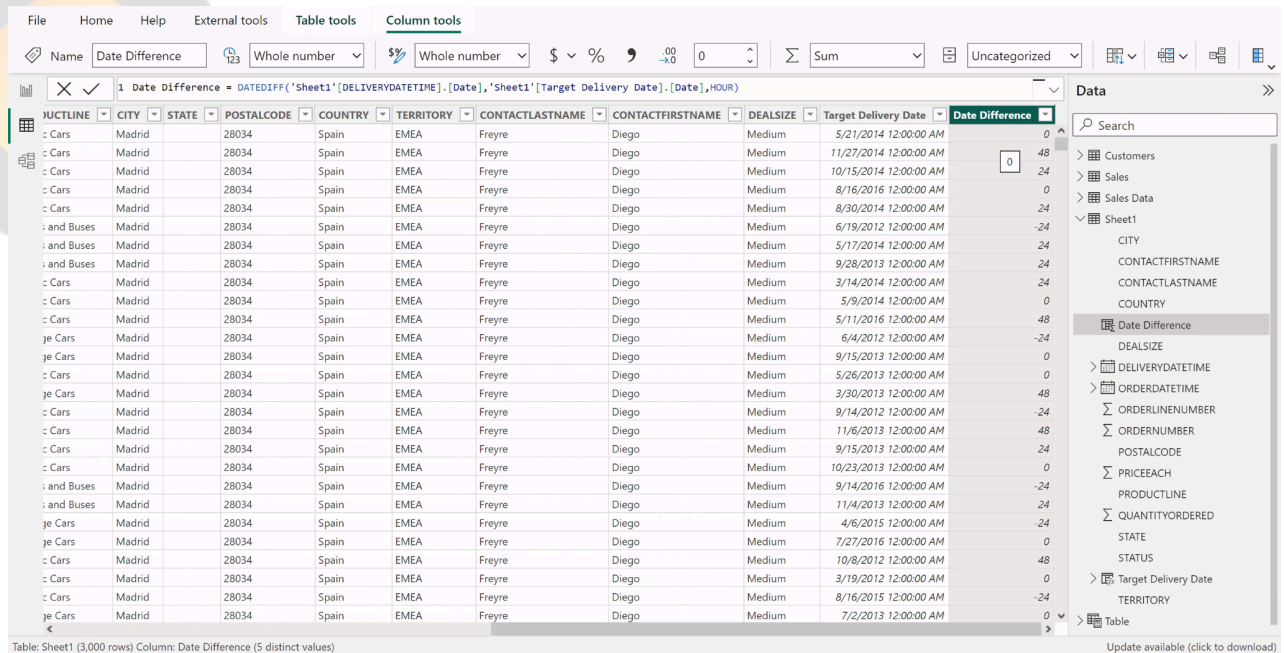
Target Delivery Date = DATEADD(Sheet1[ORDERDATETIME].[Date],2,DAY)



The screenshot shows a data table with columns: STATUS, PRODUCTLINE, CITY, STATE, POSTALCODE, COUNTRY, TERRITORY, CONTACTLASTNAME, CONTACTFIRSTNAME, DEALSIZE, and Target Delivery Date. The formula for the Target Delivery Date column is: `DATEADD(Sheet1[ORDERDATETIME].[Date],2,DAY)`. The table contains 3,000 rows of data, showing various vehicle orders and their delivery dates.

Note: Here Target Delivery Date has added 2 days without considering the timestamp of Order Date. DATEADD just adds 2 days without considering the timestamp. A better way here would be to add 48 hours without using the DATEADD function, but here we just intend to demonstrate the use of DATEADD.

Now, we can simply use DATEDIFF to find the difference between the target delivery date and the actual delivery date, as shown below.



PRODUCTLINE	CITY	STATE	POSTALCODE	COUNTRY	TERRITORY	CONTACTLASTNAME	CONTACTFIRSTNAME	DEALSIZE	Target Delivery Date	Date Difference
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	5/21/2014 12:00:00 AM	0
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	11/27/2014 12:00:00 AM	48
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	10/15/2014 12:00:00 AM	24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	8/16/2016 12:00:00 AM	0
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	8/30/2014 12:00:00 AM	-24
and Buses	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	6/19/2012 12:00:00 AM	-24
and Buses	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	5/17/2014 12:00:00 AM	24
and Buses	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	9/28/2013 12:00:00 AM	24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	3/14/2014 12:00:00 AM	24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	5/9/2014 12:00:00 AM	0
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	5/11/2016 12:00:00 AM	48
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	6/4/2012 12:00:00 AM	-24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	9/15/2013 12:00:00 AM	0
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	5/26/2013 12:00:00 AM	0
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	3/30/2013 12:00:00 AM	48
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	9/14/2012 12:00:00 AM	-24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	11/6/2013 12:00:00 AM	48
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	9/15/2013 12:00:00 AM	24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	10/23/2013 12:00:00 AM	0
and Buses	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	9/14/2016 12:00:00 AM	-24
and Buses	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	11/4/2013 12:00:00 AM	24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	4/6/2015 12:00:00 AM	-24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	7/27/2016 12:00:00 AM	0
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	10/8/2012 12:00:00 AM	48
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	3/19/2012 12:00:00 AM	0
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	8/16/2015 12:00:00 AM	-24
Cars	Madrid		28034	Spain	EMEA	Freyre	Diego	Medium	7/2/2013 12:00:00 AM	0

If the difference is positive, then we have kept the 2 day delivery promise otherwise we have broken the delivery promise.

2. DAX Time Intelligence Functions

Time Intelligence functions in DAX are designed to perform calculations and analysis based on dates and times within Power BI. When we have columns that include date and timestamp, these functions become very useful. These functions enable you to perform calculations, comparisons, and identify trends within your data based on specific time periods.

Uses:

- **Understanding trends and seasonality:** Time Intelligence functions help with identifying patterns and trends over periods of time. They help us understand growth or decline in parameters like sales, rainfall, temperature, etc. Further, they help us compare values with previous years, quarters, months, days etc.
- **Helping companies with performance monitoring:** They help companies achieve targets through effective monitoring and analysis of the current trend/pattern and the progress towards goals. They help in meeting the targets through Evaluating progress and forecasting.
- **Sharper reporting and drill downs:** They help in narrowing down data using functions easily. We can filter for specific dates/timestamps, or the period between

dates/timestamps easily using Time Intelligence functions. This helps in improving reporting efficiency and enables us to make clear and concise reports.

- **Simpler calculations:** They make date/time calculations simpler through many powerful functions.

Below are some of the commonly used Time Intelligence functions:

a. TOTALYTD

It calculates the year-to-date total of the expression for the specified dates.

Syntax: TOTALYTD(<expression>, <dates>)

b. TOTALMTD

It calculates the month-to-date total of the expression for the specified dates.

Syntax: TOTALMTD(<expression>, <dates>)

c. TOTALQTD

It calculates the quarter-to-date total of the expression for the specified dates.

Syntax: TOTALQTD(<expression>, <dates>)

d. DATESYTD

It returns a table of dates from the beginning of the year up to the specified date.

Syntax: DATESYTD(<dates>, [<year_end_date>])

e. DATESMTD

It returns a table of dates from the beginning of the month up to the specified date.

Syntax: DATESMTD(<dates>)

f. DATESQTD

It returns a table of dates from the beginning of the quarter up to the specified date.

Syntax: DATESQTD(<dates>)

g. DATESINPERIOD

It returns a table of dates for a specified number of intervals before or after a given start date.

Syntax: DATESINPERIOD(<dates>, <start_date>, <number_of_intervals>, <interval>)

h. DATESBETWEEN

It returns a table of dates that fall between the specified start date and end date, inclusive of both endpoints.

Syntax: DATESBETWEEN(<dates>, <start_date>, <end_date>)

i. SAMEPERIODLASTYEAR

It returns a table of dates for the same period in the previous year.

Syntax: SAMEPERIODLASTYEAR(<dates>)

j. PARALLELPERIOD

It returns a table of dates for the same period in a previous or future period.

Syntax: PARALLELPERIOD(<dates>, <number_of_intervals>, <interval>)

k. PREVIOUSYEAR

It returns the value from the previous year, relative to the current date.

Syntax: PREVIOUSYEAR(<dates>)

l. PREVIOUSQUARTER

It returns the value from the previous quarter, relative to the current date.

Syntax: PREVIOUSQUARTER(<dates>)

3. Advanced DAX Functions

a. CALCULATE

It enables us to do a calculation and modify the context in which other functions or expressions are evaluated. It allows us to apply additional filters, change row or column context, and manipulate the calculation context in various ways.

Syntax: CALCULATE(<expression>, <filter1>, <filter2>, ...)

Example:

```
CALCULATE(
    SUM(Sales[Amount]),
    Sales[Region] = "North"
)
```

This can be used to find the total sales amount for the North Region.

b. FILTER

FILTER(<table>, <filter_expression>)

It returns a filtered table based on some condition. It iterates over each row of the table and evaluates a logical expression for each row. If the expression evaluates to TRUE for a particular row, that row is included in the resulting table; otherwise, it is excluded.

Syntax:

Example: In our [dataset](#), we can use this:

```
FILTER(Sheet1, Sheet1[QUANTITYORDERED]>40)
```

This will give us a table with the rows in which the number of items ordered is more than 40.

The FILTER function is commonly used in combination with other DAX functions, such as CALCULATE, to apply filters to calculations and aggregations. For example:

```
CALCULATE(
    SUM(Sales[Amount]),
    FILTER(Sales, Sales[Product] = "ProductA")
)
```


This will give us the total sales for ProductA.

Note: FILTER can be used to filter down a table to create a new table. For example, in our dataset, we have a sales table with data from 2012 to 2017 but suppose we want only 2014 data. We can create a new calculated table using the following:

2014_Sales = FILTER(Sheet1, Sheet1[ORDERDATETIME].[Year]="2014")

In this manner we can filter an existing table to create a new, more specific table.

c. SUMMARIZE

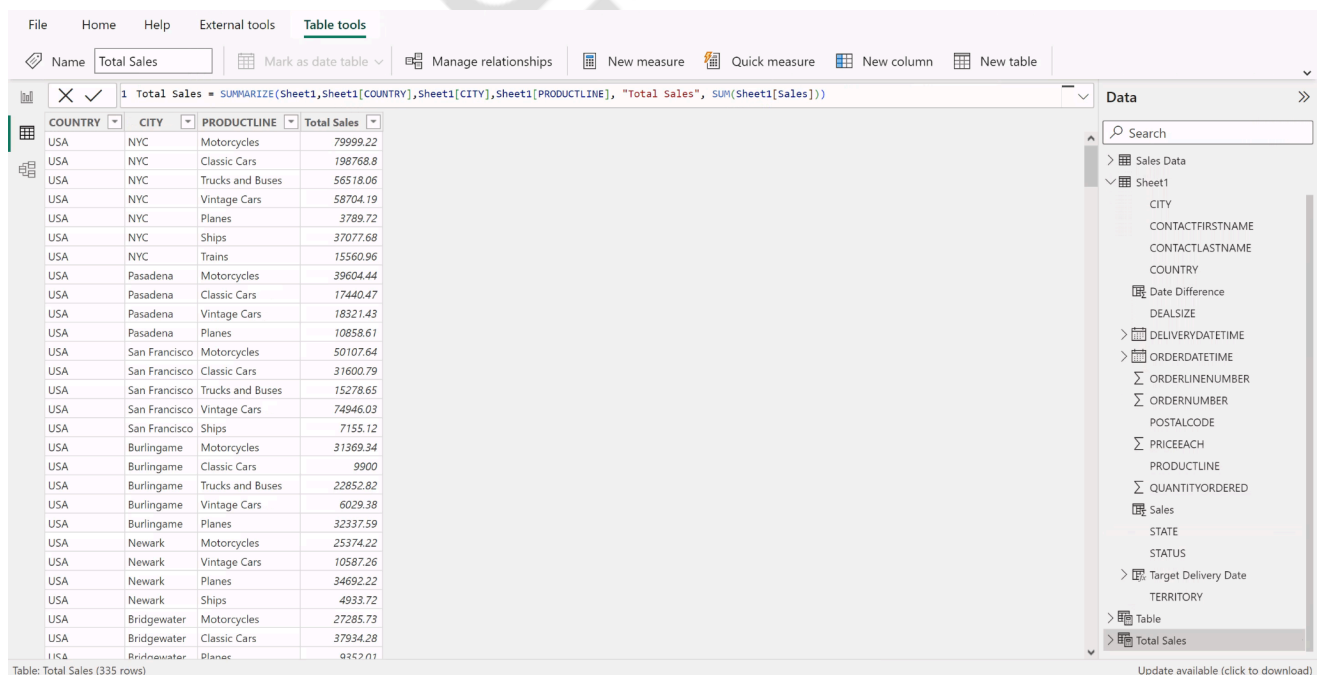
It is used to create a summary table by grouping data based on one or more columns and aggregating values using specified expressions.

Syntax: SUMMARIZE(<table>, <group_by_column1>, <group_by_column2>, ..., <name1>, <name2>, ..., <expression1>, <expression2>, ...)

Example: In our [dataset](#), we can use the SUMMARIZE function in this manner:

If we want to see the total sales for each country, each city and each product line, we can use this function as shown below:

Total Sales =
*SUMMARIZE(Sheet1,Sheet1[COUNTRY],Sheet1[CITY],Sheet1[PRODUCTLINE],
 "Total Sales", SUM(Sheet1[Sales]))*



COUNTRY	CITY	PRODUCTLINE	Total Sales
USA	NYC	Motorcycles	79999.22
USA	NYC	Classic Cars	198768.8
USA	NYC	Trucks and Buses	56518.06
USA	NYC	Vintage Cars	58704.19
USA	NYC	Planes	3789.72
USA	NYC	Ships	37077.68
USA	NYC	Trains	15560.96
USA	Pasadena	Motorcycles	39604.44
USA	Pasadena	Classic Cars	17440.47
USA	Pasadena	Vintage Cars	18321.43
USA	Pasadena	Planes	10858.61
USA	San Francisco	Motorcycles	50107.64
USA	San Francisco	Classic Cars	31600.79
USA	San Francisco	Trucks and Buses	15278.65
USA	San Francisco	Vintage Cars	74946.03
USA	San Francisco	Ships	7155.12
USA	Burlingame	Motorcycles	31369.34
USA	Burlingame	Classic Cars	9900
USA	Burlingame	Trucks and Buses	22852.82
USA	Burlingame	Vintage Cars	6029.38
USA	Burlingame	Planes	32337.59
USA	Newark	Motorcycles	25374.22
USA	Newark	Vintage Cars	10587.26
USA	Newark	Planes	34692.22
USA	Newark	Ships	4933.72
USA	Bridgewater	Motorcycles	27285.73
USA	Bridgewater	Classic Cars	37934.28
USA	Bridgewater	Planes	9252.01

d. ADDCOLUMNS

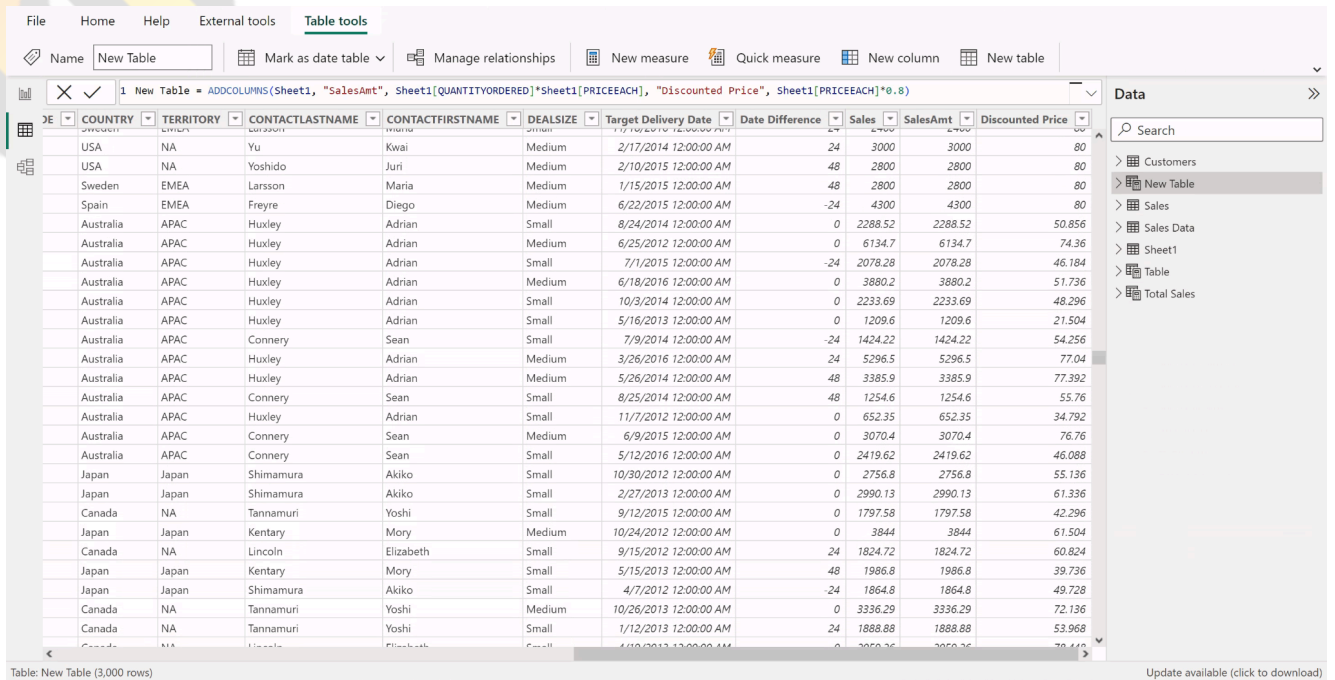
It creates a new table by adding one or more columns to an existing table or table expression. It allows you to derive new columns based on calculations or expressions applied to existing columns within the table.

Syntax: ADDCOLUMNS(<table>, <name>, <expression1>, <name>, <expression2>, ...)

Example: In our [dataset](#), we can create a new table with the Sales Column, which was originally not there in the dataset. Let's suppose the company is running a 20% discount scheme. In that case, we can also add the discounted price column to the table.

This can be done using this formula, as shown below:

*New Table = ADDCOLUMNS(Sheet1, "SalesAmt",
Sheet1[QUANTITYORDERED]*Sheet1[PRICEEACH], "Discounted Price",
Sheet1[PRICEEACH]*0.8)*



COUNTRY	TERRITORY	CONTACTLASTNAME	CONTACTFIRSTNAME	DEALSIZE	Target Delivery Date	Date Difference	Sales	SalesAmt	Discounted Price
USA	NA	Yu	Kwai	Medium	2/17/2014 12:00:00 AM	24	3000	3000	80
USA	NA	Yoshido	Juri	Medium	2/10/2015 12:00:00 AM	48	2800	2800	80
Sweden	EMEA	Larsson	Maria	Medium	1/15/2015 12:00:00 AM	48	2800	2800	80
Spain	EMEA	Freyre	Diego	Medium	6/22/2015 12:00:00 AM	-24	4300	4300	80
Australia	APAC	Huxley	Adrian	Small	8/24/2014 12:00:00 AM	0	2288.52	2288.52	50.856
Australia	APAC	Huxley	Adrian	Medium	6/25/2012 12:00:00 AM	0	6134.7	6134.7	74.36
Australia	APAC	Huxley	Adrian	Small	7/1/2015 12:00:00 AM	-24	2078.28	2078.28	46.184
Australia	APAC	Huxley	Adrian	Medium	6/18/2016 12:00:00 AM	0	3880.2	3880.2	51.736
Australia	APAC	Huxley	Adrian	Small	10/3/2014 12:00:00 AM	0	2233.69	2233.69	48.296
Australia	APAC	Huxley	Adrian	Small	5/16/2013 12:00:00 AM	0	1209.6	1209.6	21.504
Australia	APAC	Connery	Sean	Small	7/9/2014 12:00:00 AM	-24	1424.22	1424.22	54.256
Australia	APAC	Huxley	Adrian	Medium	3/26/2016 12:00:00 AM	24	5296.5	5296.5	77.04
Australia	APAC	Huxley	Adrian	Medium	5/26/2014 12:00:00 AM	48	3385.9	3385.9	77.392
Australia	APAC	Connery	Sean	Small	8/25/2014 12:00:00 AM	48	1254.6	1254.6	55.76
Australia	APAC	Huxley	Adrian	Small	11/7/2012 12:00:00 AM	0	652.35	652.35	34.792
Australia	APAC	Connery	Sean	Medium	6/9/2015 12:00:00 AM	0	3070.4	3070.4	76.76
Australia	APAC	Connery	Sean	Small	5/12/2016 12:00:00 AM	0	2419.62	2419.62	46.088
Japan	Japan	Shimamura	Akiko	Small	10/30/2012 12:00:00 AM	0	2756.8	2756.8	55.136
Japan	Japan	Shimamura	Akiko	Small	2/27/2013 12:00:00 AM	0	2990.13	2990.13	61.336
Canada	NA	Tannamuri	Yoshi	Small	9/12/2015 12:00:00 AM	0	1797.58	1797.58	42.296
Japan	Japan	Kentary	Mory	Medium	10/24/2012 12:00:00 AM	0	3844	3844	61.504
Canada	NA	Lincoln	Elizabeth	Small	9/15/2012 12:00:00 AM	24	1824.72	1824.72	60.824
Japan	Japan	Kentary	Mory	Small	5/15/2013 12:00:00 AM	48	1986.8	1986.8	39.736
Japan	Japan	Shimamura	Akiko	Small	4/7/2012 12:00:00 AM	-24	1864.8	1864.8	49.728
Canada	NA	Tannamuri	Yoshi	Medium	10/26/2013 12:00:00 AM	0	3336.29	3336.29	72.136
Canada	NA	Tannamuri	Yoshi	Small	1/12/2013 12:00:00 AM	24	1888.88	1888.88	53.968
Canada	NA	Tannamuri	Yoshi	Small	4/10/2013 12:00:00 AM	0	2050.36	2050.36	70.416

e. GROUPBY

In DAX, GROUPBY is a clause used within ADDCOLUMNS or SUMMARIZE functions to group data from a table based on specified columns and perform aggregation on each group. It enables us to create summary tables by defining grouping criteria and using aggregate functions to aggregate values for each group.

*Syntax: GROUPBY (<table>, [<groupBy_columnName>],
[<groupBy_columnName>], ..., [<name1>], [<expression1>], [<name2>],
[<expression2>], ...)*

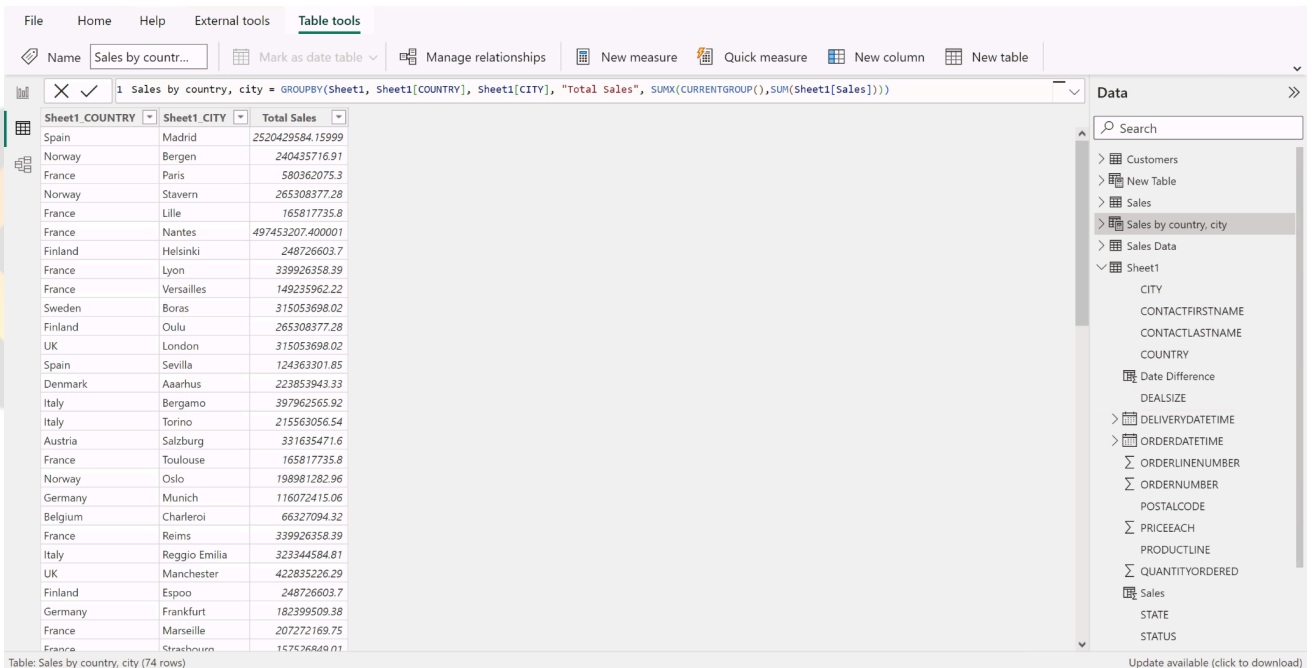
Example: In our [dataset](#), we can use the GROUP BY function to create a new table with total sales in each country and city. For this, we will group by country and city and will find the sum of sales.

Note: we are using another function: CURRENTGROUP() here.

CURRENTGROUP refers to the current group that has been created and is being evaluated. It allows you to access the values of columns within the current group and perform calculations or comparisons based on those values. It returns the rows in the table argument of the GROUPBY function which correspond to one group of values of the groupBy_columnName arguments.

The formula used here is given below:

Sales by country, city = GROUPBY(Sheet1, Sheet1[COUNTRY], Sheet1[CITY], "Total Sales", SUMX(CURRENTGROUP(),Sheet1[Sales]))



The screenshot shows the Power BI Desktop interface. The formula bar at the top contains the DAX formula: `1 Sales by country, city = GROUPBY(Sheet1, Sheet1[COUNTRY], Sheet1[CITY], "Total Sales", SUMX(CURRENTGROUP(),Sheet1[Sales]))`. Below the formula bar, a table is displayed with the following data:

Sheet1_COUNTRY	Sheet1_CITY	Total Sales
Spain	Madrid	2520429584.15999
Norway	Bergen	240435716.91
France	Paris	580362075.3
Norway	Stavern	265308377.28
France	Lille	165817735.8
France	Nantes	497453207.400001
Finland	Helsinki	248726603.7
France	Lyon	339926358.39
France	Versailles	149235962.22
Sweden	Boras	315053698.02
Finland	Oulu	265308377.28
UK	London	315053698.02
Spain	Sevilla	124363301.85
Denmark	Aarhus	223853943.33
Italy	Bergamo	397962565.92
Italy	Torino	215563056.54
Austria	Salzburg	331635471.6
France	Toulouse	165817735.8
Norway	Oslo	188981282.96
Germany	Munich	116072415.06
Belgium	Charleroi	66327094.32
France	Reims	339926358.39
Italy	Reggio Emilia	323344584.81
UK	Manchester	422835226.29
Finland	Espoo	248726603.7
Germany	Frankfurt	182399509.38
France	Marseille	207272169.75
France	Strasbourg	157526840.01

The table has 74 rows. The right-hand pane shows the 'Data' view with a search bar and a list of fields including CUSTOMERS, SALES, and various columns from Sheet1.

Note: Here instead of a sum, we need to use the SUMX because otherwise it would SUM over the entire table and not the current group that we have created (using country and city).

f. VAR and RETURN

It is used to define variables in DAX. Variables allow us to store intermediate results and we can reuse them in the same DAX expression.

Uses of VAR:

- Improves readability
- Enhances performance by reducing redundant calculations
- Simplifies complex calculations

Syntax: VAR <variable_name> = <expression>

RETURN <result_expression>

Example: In our dataset, we can use VAR as follows:

DiscountedPrice =

VAR BasePrice =

'Sheet1'[PRICEEACH]

VAR Discount =

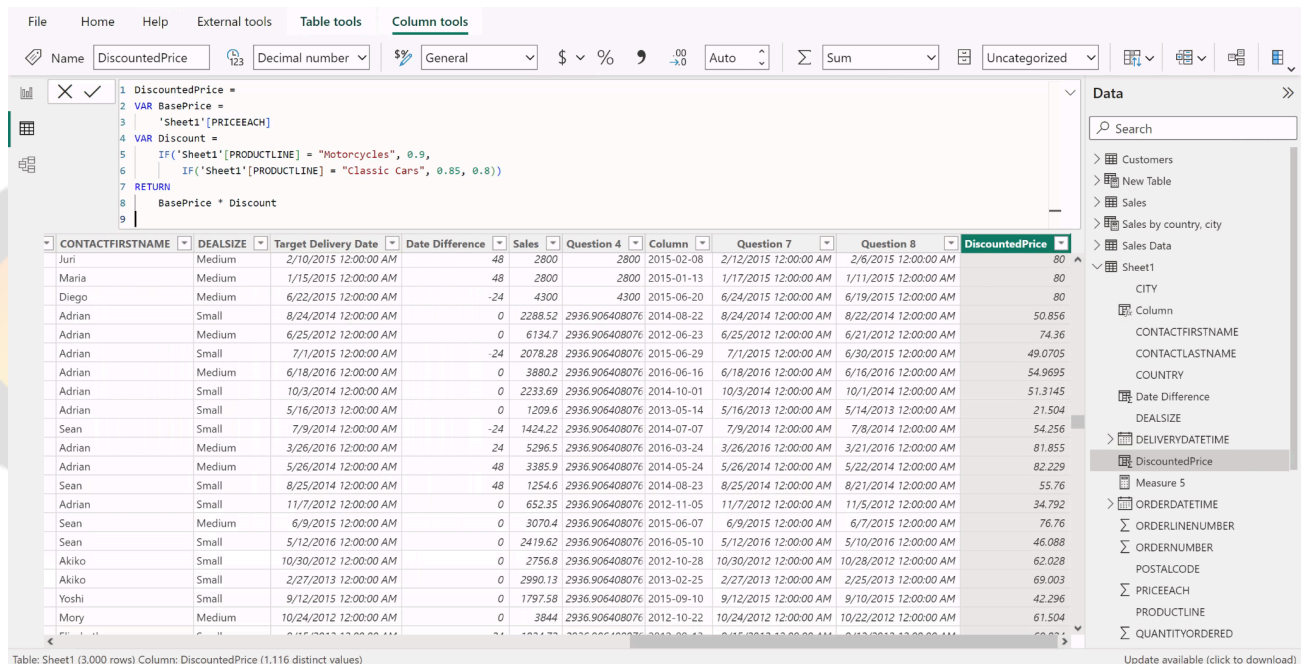
IF('Sheet1'[PRODUCTLINE] = "Motorcycles", 0.9,

IF('Sheet1'[PRODUCTLINE] = "Classic Cars", 0.85, 0.8))

RETURN

*BasePrice * Discount*

This can be seen in the image below from our dataset.



The screenshot shows a data tool interface with a formula editor and a data table. The formula editor displays the following code:

```

1 DiscountedPrice =
2 VAR BasePrice =
3   'Sheet1'[PRICEEACH]
4 VAR Discount =
5   IF('Sheet1'[PRODUCTLINE] = "Motorcycles", 0.9,
6     IF('Sheet1'[PRODUCTLINE] = "Classic Cars", 0.85, 0.8))
7 RETURN
8   BasePrice * Discount
9

```

The data table below shows columns: CONTACTFIRSTNAME, DEALSIZE, Target Delivery Date, Date Difference, Sales, Question 4, Column, Question 7, Question 8, and DiscountedPrice. The DiscountedPrice column contains values calculated based on the formula.

CONTACTFIRSTNAME	DEALSIZE	Target Delivery Date	Date Difference	Sales	Question 4	Column	Question 7	Question 8	DiscountedPrice
Juri	Medium	2/10/2015 12:00:00 AM	48	2800	2800	2015-02-08	2/12/2015 12:00:00 AM	2/6/2015 12:00:00 AM	80
Maria	Medium	1/15/2015 12:00:00 AM	48	2800	2800	2015-01-13	1/17/2015 12:00:00 AM	1/11/2015 12:00:00 AM	80
Diego	Medium	6/22/2015 12:00:00 AM	-24	4300	4300	2015-06-20	6/24/2015 12:00:00 AM	6/19/2015 12:00:00 AM	80
Adrian	Small	8/24/2014 12:00:00 AM	0	2288.52	2936.906408076	2014-08-22	8/24/2014 12:00:00 AM	8/22/2014 12:00:00 AM	50.856
Adrian	Medium	6/25/2012 12:00:00 AM	0	6134.7	2936.906408076	2012-06-23	6/25/2012 12:00:00 AM	6/21/2012 12:00:00 AM	74.36
Adrian	Small	7/1/2015 12:00:00 AM	-24	2078.28	2936.906408076	2015-06-29	7/1/2015 12:00:00 AM	6/30/2015 12:00:00 AM	49.0705
Adrian	Medium	6/18/2016 12:00:00 AM	0	3880.2	2936.906408076	2016-06-16	6/18/2016 12:00:00 AM	6/16/2016 12:00:00 AM	54.9695
Adrian	Small	10/3/2014 12:00:00 AM	0	2233.69	2936.906408076	2014-10-01	10/3/2014 12:00:00 AM	10/1/2014 12:00:00 AM	51.3145
Adrian	Small	5/16/2013 12:00:00 AM	0	1209.6	2936.906408076	2013-05-14	5/16/2013 12:00:00 AM	5/14/2013 12:00:00 AM	21.504
Sean	Small	7/9/2014 12:00:00 AM	-24	1424.22	2936.906408076	2014-07-07	7/9/2014 12:00:00 AM	7/8/2014 12:00:00 AM	54.256
Adrian	Medium	3/26/2016 12:00:00 AM	24	5296.5	2936.906408076	2016-03-24	3/26/2016 12:00:00 AM	3/21/2016 12:00:00 AM	81.855
Adrian	Medium	5/26/2014 12:00:00 AM	48	3385.9	2936.906408076	2014-05-24	5/26/2014 12:00:00 AM	5/22/2014 12:00:00 AM	82.229
Sean	Small	8/25/2014 12:00:00 AM	48	1254.6	2936.906408076	2014-08-23	8/25/2014 12:00:00 AM	8/21/2014 12:00:00 AM	55.76
Adrian	Small	11/7/2012 12:00:00 AM	0	652.35	2936.906408076	2012-11-05	11/7/2012 12:00:00 AM	11/5/2012 12:00:00 AM	34.792
Sean	Medium	6/9/2015 12:00:00 AM	0	3070.4	2936.906408076	2015-06-07	6/9/2015 12:00:00 AM	6/7/2015 12:00:00 AM	76.76
Sean	Small	5/12/2016 12:00:00 AM	0	2419.62	2936.906408076	2016-05-10	5/12/2016 12:00:00 AM	5/10/2016 12:00:00 AM	46.088
Akiko	Small	10/30/2012 12:00:00 AM	0	2756.8	2936.906408076	2012-10-28	10/30/2012 12:00:00 AM	10/28/2012 12:00:00 AM	62.028
Akiko	Small	2/27/2013 12:00:00 AM	0	2990.13	2936.906408076	2013-02-25	2/27/2013 12:00:00 AM	2/25/2013 12:00:00 AM	69.003
Yoshi	Small	9/12/2015 12:00:00 AM	0	1797.58	2936.906408076	2015-09-10	9/12/2015 12:00:00 AM	9/10/2015 12:00:00 AM	42.296
Mory	Medium	10/24/2012 12:00:00 AM	0	3844	2936.906408076	2012-10-22	10/24/2012 12:00:00 AM	10/22/2012 12:00:00 AM	61.504

Here, we are calculating the discounted price. If the product line is Motorcycles then discount is 10%, if the product line is Classic Cars then discount is 15%, otherwise the discount is 20%. Here the VAR function is used to simplify our calculation.

As demonstrated above, the **RETURN** keyword is used within calculated columns, measures, and table expressions to specify the value or result that should be returned by the expression. It marks the end of the expression and determines the final value to be used in the expression.

g. RELATED

It returns the value from a column in a related table. The data model should have the relationship defined for this function to work.

Syntax: RELATED(<column_name>)

For example, if we have sales orders in one table and product data in another table, and we want to create a calculated column for one particular product, we can use RELATED(Product[Product Name]). For each order ID this would return the product name.

Note: It is important to note here that RELATED will not work if relationships are not defined.

h. FORMAT

It converts a value to text based on the specified format.

Syntax: FORMAT(<value>, <format_string>, [<locale_name>])

Here, format_string is the string with the formatting template.

locale_name is the name of the locale.

For example: = *FORMAT(700.17, "Percent")* returns 70017.00%

i. CROSSFILTER

It is used to define the directions of the relationship between 2 columns.

Syntax: CROSSFILTER(<columnName1>, <columnName2>, <direction>)

Direction can have the following values:

None, Both: creates a bidirectional filter

OneWay: filters applied on one side filter the other side

OneWay_LeftFiltersRight: filters applied on left side filter the right side

OneWay_RightFiltersLeft: filters applied on right side filter the left side

j. ALL

It clears all the filters and returns the value. In other words, it returns all the values in the table or column, ignoring any applied filters or context.

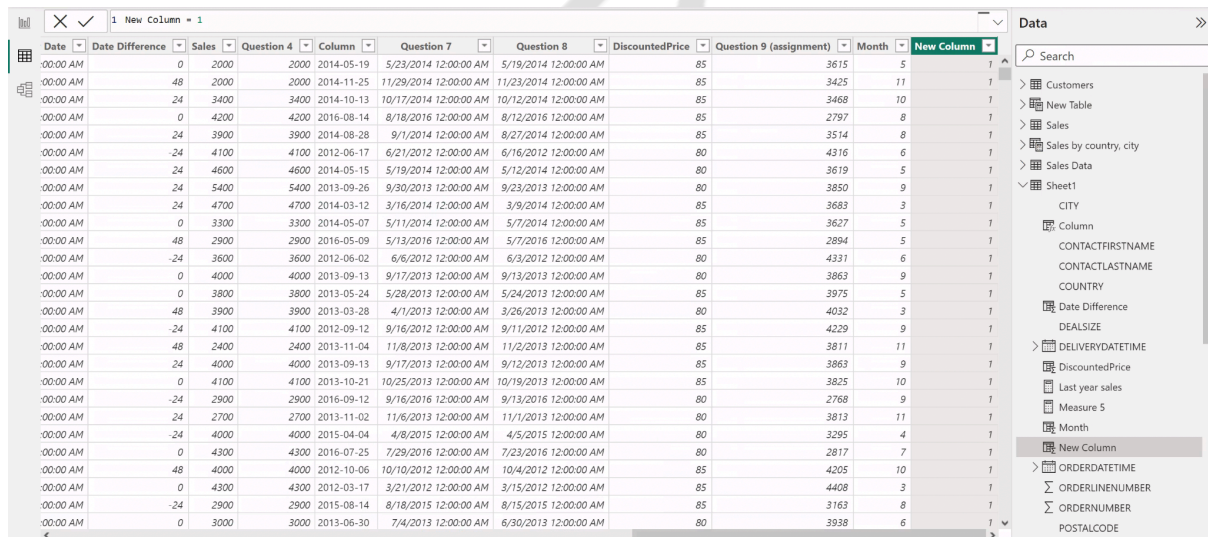
Syntax: ALL (<table>)

or

ALL (<table>[column])

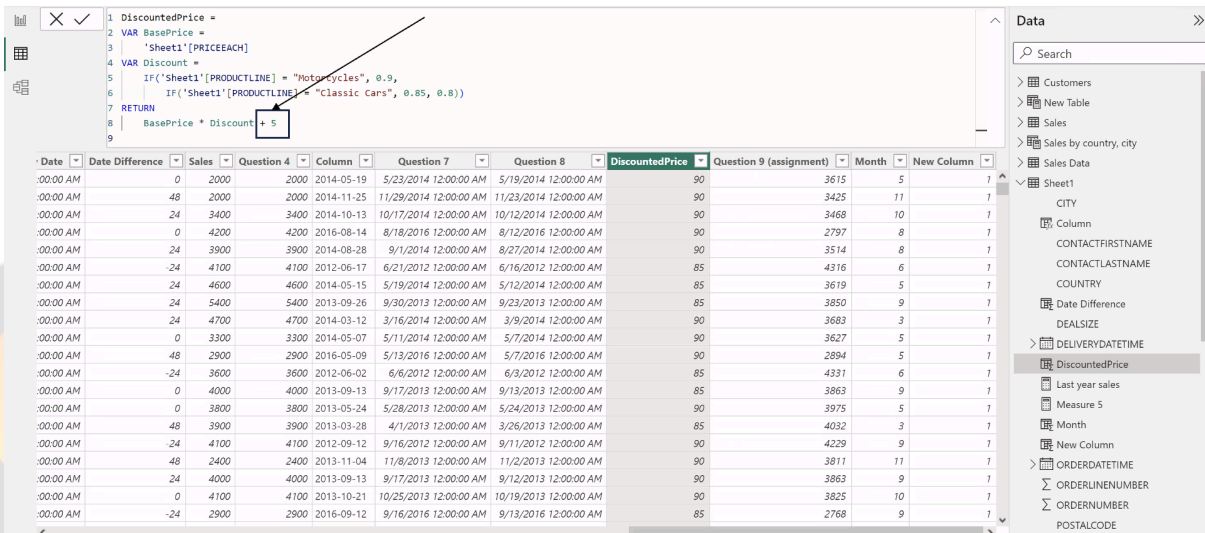
4. Other basic functionalities in DAX

DAX is an easy to understand language and very intuitive. For example, the SUM, COUNT functions work just like Excel. If we want to define a column as a particular fixed value, we can simply write: Value = 1. This is shown below:



Date	Date Difference	Sales	Question 4	Column	Question 7	Question 8	DiscountedPrice	Question 9 (assignment)	Month	New Column
0:00 AM	0	2000	2000	2014-05-19	5/23/2014 12:00:00 AM	5/19/2014 12:00:00 AM	85	3615	5	1
0:00 AM	48	2000	2000	2014-11-25	11/29/2014 12:00:00 AM	11/23/2014 12:00:00 AM	85	3425	11	1
0:00 AM	24	3400	3400	2014-10-13	10/17/2014 12:00:00 AM	10/12/2014 12:00:00 AM	85	3468	10	1
0:00 AM	0	4200	4200	2016-08-14	8/18/2016 12:00:00 AM	8/12/2016 12:00:00 AM	85	2797	8	1
0:00 AM	24	3900	3900	2014-08-28	9/1/2014 12:00:00 AM	8/27/2014 12:00:00 AM	85	3514	8	1
0:00 AM	-24	4100	4100	2012-06-17	6/21/2012 12:00:00 AM	6/16/2012 12:00:00 AM	80	4316	6	1
0:00 AM	24	4600	4600	2014-05-15	5/19/2014 12:00:00 AM	5/12/2014 12:00:00 AM	80	3619	5	1
0:00 AM	24	5400	5400	2013-09-26	9/30/2013 12:00:00 AM	9/23/2013 12:00:00 AM	80	3850	9	1
0:00 AM	24	4700	4700	2014-03-12	3/16/2014 12:00:00 AM	3/9/2014 12:00:00 AM	85	3683	3	1
0:00 AM	0	3300	3300	2014-05-07	5/11/2014 12:00:00 AM	5/7/2014 12:00:00 AM	85	3627	5	1
0:00 AM	48	2900	2900	2016-05-09	5/13/2016 12:00:00 AM	5/7/2016 12:00:00 AM	85	2894	5	1
0:00 AM	-24	3600	3600	2012-06-02	6/6/2012 12:00:00 AM	6/3/2012 12:00:00 AM	80	4331	6	1
0:00 AM	0	4000	4000	2013-09-13	9/17/2013 12:00:00 AM	9/13/2013 12:00:00 AM	80	3863	9	1
0:00 AM	0	3800	3800	2013-05-24	5/28/2013 12:00:00 AM	5/24/2013 12:00:00 AM	85	3975	5	1
0:00 AM	48	3900	3900	2013-03-28	4/1/2013 12:00:00 AM	3/26/2013 12:00:00 AM	80	4032	3	1
0:00 AM	-24	4100	4100	2012-09-12	9/16/2012 12:00:00 AM	9/11/2012 12:00:00 AM	85	4229	9	1
0:00 AM	48	2400	2400	2013-11-04	11/8/2013 12:00:00 AM	11/2/2013 12:00:00 AM	85	3811	11	1
0:00 AM	24	4000	4000	2013-09-13	9/17/2013 12:00:00 AM	9/12/2013 12:00:00 AM	85	3863	9	1
0:00 AM	0	4100	4100	2013-10-21	10/25/2013 12:00:00 AM	10/19/2013 12:00:00 AM	85	3825	10	1
0:00 AM	-24	2900	2900	2016-09-12	9/16/2016 12:00:00 AM	9/13/2016 12:00:00 AM	80	2768	9	1
0:00 AM	24	2700	2700	2013-11-02	11/6/2013 12:00:00 AM	11/1/2013 12:00:00 AM	80	3813	11	1
0:00 AM	-24	4000	4000	2015-04-04	4/8/2015 12:00:00 AM	4/5/2015 12:00:00 AM	80	3295	4	1
0:00 AM	0	4300	4300	2016-07-25	7/29/2016 12:00:00 AM	7/23/2016 12:00:00 AM	80	2817	7	1
0:00 AM	48	4000	4000	2012-10-06	10/10/2012 12:00:00 AM	10/4/2012 12:00:00 AM	85	4205	10	1
0:00 AM	0	4300	4300	2012-03-17	3/21/2012 12:00:00 AM	3/15/2012 12:00:00 AM	85	4408	3	1
0:00 AM	-24	2900	2900	2015-08-14	8/18/2015 12:00:00 AM	8/15/2015 12:00:00 AM	85	3163	8	1
0:00 AM	0	3000	3000	2013-06-30	7/4/2013 12:00:00 AM	6/30/2013 12:00:00 AM	80	3938	6	1

Similarly, if we want to add '1' to a column, we can do so by simply adding a "+1" in the DAX code of the column. For example, let's say we want to add a ₹ 5 logistics fee in the Discounted price. We can simply add a "+5" at the end of the formula in DAX.



The screenshot shows the Power BI Desktop interface. On the left, the DAX formula bar contains the following code:

```

1 DiscountedPrice =
2 VAR BasePrice =
3   'Sheet1'[PRICEEACH]
4 VAR Discount =
5   IF('Sheet1'[PRODUCTLINE] = "Motorcycles", 0.9,
6     IF('Sheet1'[PRODUCTLINE] = "Classic Cars", 0.85, 0.8))
7 RETURN
8   BasePrice * Discount + 5
9

```

An arrow points from the formula bar to the 'DiscountedPrice' column in the data table. The data table has the following columns: Date, Date Difference, Sales, Question 4, Column, Question 7, Question 8, DiscountedPrice, Question 9 (assignment), Month, and New Column. The table contains 20 rows of data.

Date	Date Difference	Sales	Question 4	Column	Question 7	Question 8	DiscountedPrice	Question 9 (assignment)	Month	New Column
:00:00 AM	0	2000	2000	2014-05-19	5/23/2014 12:00:00 AM	5/19/2014 12:00:00 AM	90	3615	5	?
:00:00 AM	48	2000	2000	2014-11-25	11/29/2014 12:00:00 AM	11/23/2014 12:00:00 AM	90	3425	11	?
:00:00 AM	24	3400	3400	2014-10-13	10/17/2014 12:00:00 AM	10/12/2014 12:00:00 AM	90	3468	10	?
:00:00 AM	0	4200	4200	2016-08-14	8/18/2016 12:00:00 AM	8/12/2016 12:00:00 AM	90	2797	8	?
:00:00 AM	24	3900	3900	2014-08-28	9/1/2014 12:00:00 AM	8/27/2014 12:00:00 AM	90	3514	8	?
:00:00 AM	-24	4100	4100	2012-06-17	6/21/2012 12:00:00 AM	6/16/2012 12:00:00 AM	85	4316	6	?
:00:00 AM	24	4600	4600	2014-05-15	5/19/2014 12:00:00 AM	5/12/2014 12:00:00 AM	85	3619	5	?
:00:00 AM	24	5400	5400	2013-09-26	9/30/2013 12:00:00 AM	9/23/2013 12:00:00 AM	85	3850	9	?
:00:00 AM	24	4700	4700	2014-03-12	3/16/2014 12:00:00 AM	3/9/2014 12:00:00 AM	90	3683	3	?
:00:00 AM	0	3300	3300	2014-05-07	5/11/2014 12:00:00 AM	5/7/2014 12:00:00 AM	90	3627	5	?
:00:00 AM	48	2900	2900	2016-05-09	5/13/2016 12:00:00 AM	5/7/2016 12:00:00 AM	90	2894	5	?
:00:00 AM	-24	3600	3600	2012-06-02	6/6/2012 12:00:00 AM	6/3/2012 12:00:00 AM	85	4331	6	?
:00:00 AM	0	4000	4000	2013-09-13	9/17/2013 12:00:00 AM	9/13/2013 12:00:00 AM	85	3863	9	?
:00:00 AM	0	3800	3800	2013-05-24	5/28/2013 12:00:00 AM	5/24/2013 12:00:00 AM	90	3975	5	?
:00:00 AM	48	3900	3900	2013-03-28	4/1/2013 12:00:00 AM	3/26/2013 12:00:00 AM	85	4032	3	?
:00:00 AM	-24	4100	4100	2012-09-12	9/16/2012 12:00:00 AM	9/11/2012 12:00:00 AM	90	4229	9	?
:00:00 AM	48	2400	2400	2013-11-04	11/8/2013 12:00:00 AM	11/2/2013 12:00:00 AM	90	3811	11	?
:00:00 AM	24	4000	4000	2013-09-13	9/17/2013 12:00:00 AM	9/12/2013 12:00:00 AM	90	3863	9	?
:00:00 AM	0	4100	4100	2013-10-21	10/25/2013 12:00:00 AM	10/19/2013 12:00:00 AM	90	3825	10	?
:00:00 AM	-24	2900	2900	2016-09-12	9/16/2016 12:00:00 AM	9/13/2016 12:00:00 AM	85	2768	9	?

We can also use any other DAX operator to do other manual calculations in a similar manner.

4. Keeping all DAX in one table

It is always helpful if we create a new table and store all DAX in that table. This keeps things organized and makes it easier for us to use measures and calculated columns during visualization.

Steps to store all DAX in one table:

1. Select Enter Data option to create a new table.
2. Load the table without entering any data
3. Right click on the newly created table and start creating new measures
4. All DAX should now be defined in this 'Measures' table.

One of the benefits of this is that if we need to change any measure/calculation, we just need to change it in the measures table instead of changing it in the source tables. For example, if we have a measure 'Sales' in 5 tables, we should simply create a measure for Total Sales in the Measures table and then use it. Now if we need to do any calculation change to the Sales column, instead of changing it in 5 different tables, we only need to change it in one table, i.e. the Measures table.